

Day 8 复习总表：数组方法进阶 + 数据驱动渲染完善

Step 项	完成事项	对应代码 / 指令	代码详解
Step 2	巩固声明式函数与箭头函数两种写法 回顾	<pre>js function multiply(a, b) { return a * b; } const squareNum = num => num * num;</pre>	• 声明式函数可以在定义前调用（提升） • 箭头函数写法更短，单行表达式可省略 <code>{}</code> 和 <code>return</code>（隐式返回）
Step 3	创建、访问、修改、遍历数组 数组基础	<pre>js let fruits = ["Apple","Banana","Cherry"]; fruits[0] = "Orange"; for (let fruit of fruits) {...}</pre>	• <code>[]</code> 创建数组，下标从 0 开始 • <code>.length</code> 是属性不是方法 • <code>for...of</code> 直接拿到元素本身，比传统 for 循环更简洁
Step 4	push/pop/map/filter/reduce 处理 数组方法 进阶	<pre>js numbers.push(8); numbers.map(num => num*2); numbers.reduce((acc,cur)=>acc+cur, 0);</pre>	• <code>push</code>/<code>pop</code> 修改原数组（push=加入末尾，pop=删除末尾） • <code>map</code>/<code>filter</code> 不修改原数组，返回新数组 • <code>reduce</code> 把整个数组"压缩"成一个值，<code>acc</code> 是累加器
Step 5	用对象数组 + map 生成 HTML 卡片 数据驱动 渲染	<pre>js const projects = [{name:"...", description:"..."}]; projects.map(p => +" "+"<article>...\${p.name}... </article> "+"").join("");</pre>	• 数组里存对象，每个对象是一条完整数据 • <code>map()</code> 把数据"转换"成 HTML 字符串 • <code>.join("")</code> 把字符串数组拼成一段字符串，不加就没法直接塞进页面
Step 6	用 innerHTML 把生成的 HTML 插入页面 插入容器	<pre>js document.getElementById("project-container").innerHTML = projectCardsHTML;</pre>	• HTML 里只留一个空容器，内容完全由 JS 根据数据生成 • <code>innerHTML</code> 赋值会整体替换容器内容
Step 7	防御性编程 + 自动获取当前年份 动态页脚 年份	<pre>js if (footerYear) { const year = new Date().getFullYear(); footerYear.innerHTML += "+" + `© \${year} ...` + "+" + ; }</pre>	• <code>if (footerYear)</code> 防止元素不存在时报错中断脚本 • <code>new Date().getFullYear()</code> 自动获取当前年份，不用每年手动改
Step 8	保存本日数组方法+数据驱动渲染的 Git 提交 改动	<pre>bash git add index.html script.js</pre>	• 只暂存真正改动的两个文件 • <code>feat:</code> 前缀表示新功能，信息说明"渲染卡片+页脚年份"两件事

Step 项	完成事项	对应代码 / 指令	代码详解
		<code>git commit -m "feat: render project cards from data array and dynamic footer year"</code>	
Step 9	4 道概念题巩固理解 自我检查	考察问题： 1. <code>map()</code> vs <code>push()</code> 2. <code>.join("")</code> 为什么必须加 3. <code>if (footerYear)</code> 的保护作用 4. 新增项目该改哪个文件	• <code>push()</code>/<code>pop()</code> 是易混对：push=加入末尾，pop=删除末尾，都会改原数组；map/filter/reduce 不改原数组 • <code>.join("")</code> 把字符串数组拼接成可插入页面的完整字符串 • 防御性编程：先判断元素是否存在，避免对 <code>null</code> 操作报错 • 数据驱动渲染下，加新项目只需改 <code>script.js</code> 的数组

本日核心概念速查

概念	一句话记忆
<code>push()</code> / <code>pop()</code>	push=放进去（末尾加），pop=弹出去（末尾删），都改原数组
<code>map()</code> / <code>filter()</code> / <code>reduce()</code>	都不改原数组：map 转换、filter 筛选、reduce 压缩成一个值
<code>.join("")</code>	把 map 生成的字符串数组，拼成一段可插入页面的字符串
<code>if (元素)</code> 防御性判断	<code>querySelector</code> 找不到元素返回 <code>null</code> ，操作前先判断，避免脚本中断
数据驱动渲染	页面内容由数组决定，改数据不用改 HTML